# Tuples

## 1. What areTuples ?

- allow us to store multiple items in a single variable.

They are :

- ordered (maintain the order of elements),
- immutable (items cannot be changed after creation),
- allow duplicate values
- dynamic(can store elements of various data types and other sequence types).

### 1.1 Creating a Tuple Using Parentheses

A tuple is created by enclosing comma-separated items inside rounded brackets.

#### Syntax

```python
tuple_name = (item1, item2, item3)
```

#### Example

```python
empty_tuple = ()
int_tuple = (1, 2, 3, 4, 5)
mixed_tuple = (1, 'rest', [ 2, 4, 6 ], {2.3, 4.5, 6.7}, True)
print(type(empty_tuple))
print(int_tuple)
print(mixed_tuple)
```

#### Output

```
<class 'tuple'>
(1, 2, 3, 4, 5)
(1, 'rest', [ 2, 4, 6 ], {2.3, 4.5, 6.7}, True)
```

## 1.2 Creating a Tuple Using the tuple() Constructor

Create a tuple by passing an iterable inside the built-in tuple() constructor.

**Syntax**

```
Python
tuple_name = tuple(iterable)
```

**Example**

```
Python
lst = ['r', 't', 's']
method_tuple = tuple(lst)
print(method_tuple)
print(type(method_tuple))
```

**Output**

```
('r', 't', 's')
<class 'tuple'>
```

## 1.3 Creating a Single Item Tuple

A single item tuple must be created by enclosing one item inside parentheses followed by a comma.

**Syntax**

```python
tuple_name = (item,)
```

**Example**

```python
single_tuple = ('rest',)
print(type(single_tuple))
print(single_tuple)
```

**Output**

```
<class 'tuple'>
('rest',)
```

## 1.4 Packing and Unpacking

- Packing is collecting multiple values in a single variable without using parentheses.
- Unpacking assigns tuple items to individual variables. You can use an asterisk (*) to pack leftover middle elements into a list.

**Syntax**

```python
var1, var2 = tuple_name
```

**Example**

```python
packing_tuple = 1, 2, 3
a, b, c = packing_tuple
```

```
print(a, b, c)
packed_tup = (1, 2, 3, 4, 5, 6, 7)
f, *m, l = packed_tup
print(f)
print(m)
print(l)
```

**Output**

```
1 2 3
1
[ 2, 3, 4, 5, 6 ]
7
```

## 1.5 Finding Tuple Length

We can find the length of the tuple using the len() function.

**Syntax**

Python
```
len(tuple_name)
```

**Example**

Python
```
int_tuple = (1, 2, 3, 4, 5)
print(len(int_tuple))
```

**Output**

```
5
```

# 2. Accessing and Slicing Tuples

## 2.1 Positive Indexing

- Every item in a tuple has an index starting from 0.
- Access an item of a tuple by using its index number inside the index operator.

**Syntax**

```python
tuple_name[index]
```

**Example**

```python
t_1 = (2, 4, 1, 6, 5)
print(t_1[0])
```

**Output**

```
2
```

## 2.2 Negative Indexing

- Index values can be negative, with the very last item having the index value of -1.

**Syntax**

```python
tuple_name[-index]
```

**Example**

```python
t_1 = (2, 4, 1, 6, 5)
print(t_1[-1])
```

**Output**

```
5
```

## 2.3 Slicing a Tuple

- Specifying a range of items using the colon operator.
- The start value is included, the end value is excluded.

**Syntax**

```python
tuple_name[start:end:step]
```

**Example**

```python
t_1 = (2, 4, 1, 6, 5)
print(t_1[1:])
print(t_1[1:3])
print(t_1[::-1])
```

**Output**

```
(4, 1, 6, 5)
(4, 1)
(5, 6, 1, 4, 2)
```

## 2.4 Finding an Item Index in a Tuple

We search for a certain item using the index() method, which returns the exact position of its first occurrence.

**Syntax**

```python
tuple_name.index(value)
```

**Example**

```python
tup_imm = ('r', 'y', 'p', 'i')
print(tup_imm.index('p'))
```

**Output**

```
2
```

## 2.5 Checking if an Item Exists

We check whether an item exists in a tuple by using the in operator, which returns a boolean True or False.

**Syntax**

```python
value in tuple_name
```

**Example**

```python
t_1 = (2, 4, 1, 6, 5)
print(6 in t_1)
print(9 in t_1)
```

**Output**

```
True
False
```

# 3. Modifying Tuples

## 3.1 The List Conversion

Because a tuple is immutable, we must convert the tuple to a list, add or modify the items, and then convert it back.

**Syntax**

```python
temp_list = list(tuple_name)
temp_list.append(new_item)
tuple_name = tuple(temp_list)
```

**Example**

```python
tup_imm = ('r', 'y', 'p', 'i')
list_tup_imm = list(tup_imm)
list_tup_imm.insert(3, 'qwedgbv')
list_tup_imm.append(45)
tup_imm = tuple(list_tup_imm)
print(tup_imm)
```

**Output**

```
('r', 'y', 'p', 'qwedgbv', 'i', 45)
```

## 3.2 Modifying Nested Items

If an item inside the tuple is itself a mutable data type (like a list), we can modify its nested values directly without converting the entire tuple.

**Syntax**

```python
tuple_name[index][nested_index] = new_value
```

**Example**

```python
tuple_change = (1, 2, 3, [ 45, 87, 34, 107 ], "String")
tuple_change[3][3] = 999
print(tuple_change)
```

**Output**

```
(1, 2, 3, [ 45, 87, 34, 999 ], 'String')
```

# 4. Removing Items from Tuples

## 4.1 Using the del Keyword

The del keyword deletes the entire tuple object from memory completely.

#### Syntax

```python
del tuple_name
```

#### Example

```python
sample_tup1 = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
del sample_tup1
```

#### Output

```

```

## 4.2 By Converting into a List

We can convert a tuple into a list, remove an item using the list remove method, and convert it back into a tuple.

**Syntax**

```python
temp_list = list(tuple_name)
temp_list.remove(value)
tuple_name = tuple(temp_list)
```

**Example**

```python
tuple1 = (0, 1, 2, 3, 4, 5)
sample_list = list(tuple1)
sample_list.remove(2)
tuple1 = tuple(sample_list)
print(tuple1)
```

**Output**

```
(0, 1, 3, 4, 5)
```

# 5. Advanced Tuple Operations

## 5.1 Iterating Through a Tuple

We can iterate sequentially through a tuple using a standard for loop.

**Syntax**

```python
for item in tuple_name:
    # Code to execute
```

**Example**

```python
int_tuple = (1, 2, 3, 4, 5)
for i in int_tuple:
    print(i, end=" ")
```

**Output**

```
1 2 3 4 5
```

## 5.2 Counting Item Occurrences

The count() method accepts any value as a parameter and returns the number of times it appears.

**Syntax**

```python
tuple_name.count(value)
```

**Example**

```python
tuple1 = (10, 20, 60, 30, 60, 40, 60)
print(tuple1.count(60))
```

**Output**

```
3
```

## 5.3 Copying a Tuple

Create a reference copy of a tuple using the assignment operator. No change in the original tuple

### Syntax

```python
tuple2 = tuple1
```

### Example

```python
tuple1 = (0, 1, 2, 3, 4, 5)
tuple2 = tuple1
tuple2=list(tuple2)
tuple2.append(6)
tuple2=tuple(tuple2)
print(tuple2)
print(tuple1)
```

### Output

```
(0, 1, 2, 3, 4, 5, 6)
(0, 1, 2, 3, 4, 5)
```

## 5.4 Concatenating Tuples

We can combine tuples using the + operator, or repeat them with the * operator.

### Syntax

```python
tuple3 = tuple1 + tuple2
```

**Example**

```python
int_tuple = (1, 2, 3, 4, 5)
mixed_tuple = (1, 'rest', [ 2, 4, 6 ], {2.3, 4.5, 6.7}, True)
concate_tuple = int_tuple + mixed_tuple
print(concate_tuple)
print(mixed_tuple*3)
```

**Output**

```
(1, 2, 3, 4, 5, 1, 'rest', [ 2, 4, 6 ], {2.3, 4.5, 6.7}, True)
(1, 'rest', [2, 4, 6], {2.3, 4.5, 6.7}, True, 1, 'rest', [2, 4, 6], {2.3, 4.5, 6.7}, True, 1, 'rest', [2, 4, 6], {2.3, 4.5, 6.7}, True)
```

## 5.5 Nested Tuples

When a tuple contains another tuple as its member, it is called a nested tuple. We access inner items using multiple index brackets.

**Syntax**

```python
tuple_name[outer_index][inner_index]
```

**Example**

```python
nest_tup = ((1, 2, 3), ('r', 'j', 'm'))
print(nest_tup)
print(nest_tup[0])
print(nest_tup[1][1:])
```

**Output**

```
((1, 2, 3), ('r', 'j', 'm'))
(1, 2, 3)
('j', 'm')
```

## 5.6 Iterating Through Nested Tuples

We can retrieve and print the items of an inner tuple by using a nested for loop.

**Syntax**

```python
for outer_item in tuple_name:
    for inner_item in outer_item:
        # Code to execute
```

**Example**

```python
nest_tup = ((1, 2, 3), ('r', 'j', 'm'))
for sub_t in nest_tup:
      for e_t in sub_t:
       print(e_t,end=' ')
      print()
```

**Output**

```
1 2 3
r j m
```